

Shouyihuo

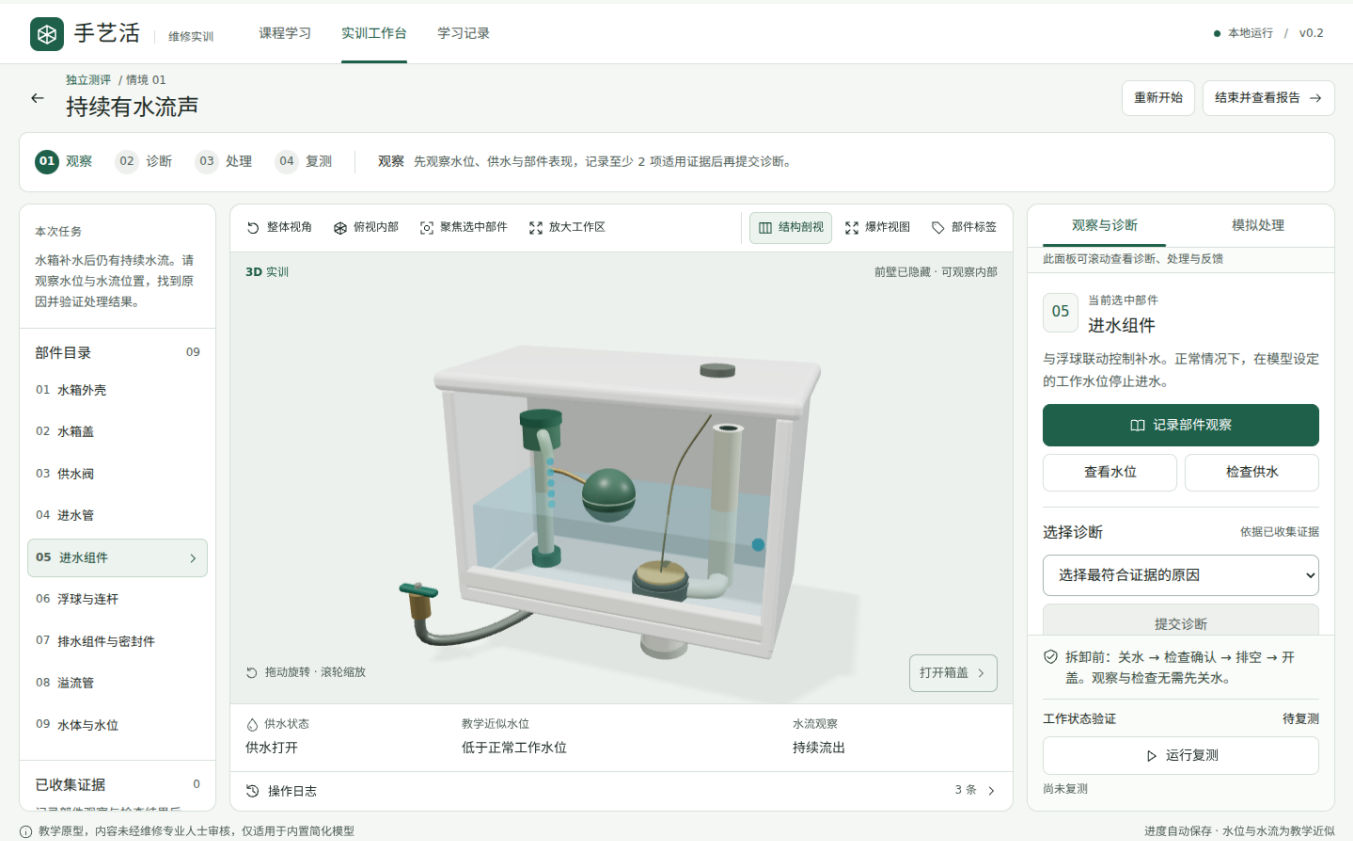
An inspectable 3D training simulation

A local-first browser prototype that connects observation, diagnosis, simulated treatment and verification to one explicit state model.

3 fault cases

3 learning modes

No backend or API key



Actual WebGL workbench, recorded on 26 September 2026. The learner interface is in Simplified Chinese; an English guide supplies exact bilingual control labels.

01 The problem: keep visual actions and rules consistent

A convincing model is insufficient if an unsafe action silently succeeds or a camera change affects the score. The prototype makes these constraints testable: rejected disassembly preserves the assembly, records the error and prevents a later safety pass.

Built-in course: a simplified gravity-fed toilet cistern with a float-controlled inlet and flapper-style drain seal. Cases cover seal failure, inlet-control failure and a closed supply valve; the last requires no replacement.

Scope: an unaudited teaching model, not repair guidance, a calibrated digital twin or a professional qualification.

One state. Several views.

Typed transitions connect the lesson, 3D scene, assessment and local records.

Typed user events

Observe, diagnose, remove, replace, retest, tick



Pure simulation reducer

Guards + water approximation + verification + score



Derived interface and validated persistence

React / Three.js read state; localStorage restores it

02 Determinism with bounded claims

The same initial state and event sequence produce the same outcome. The UI advances the model by 0.25 simulation seconds per tick; the reducer uses internal steps of at most 0.05 seconds. Water is normalised teaching state, not a measured hydraulic quantity.

03 Safety is a gate, not a score deduction

Evidence, diagnosis, process and verification receive 25, 30, 25 and 20 points. Passing also requires correct diagnosis and treatment, complete verification and no critical safety error. Even a 100-point attempt may fail.

04 Visual controls cannot bypass business rules

The 3D view reads the authoritative attempt. Cutaway, exploded view, camera focus and workspace expansion change presentation only. Model selection, the part list and the operation target share one selection.

05 Persistence preserves both progress and errors

Schema-validated localStorage keeps separate mode sessions and archives each completed attempt once by ID. The v0.1 storage key and schema remain compatible. Corrupt storage is reported instead of silently overwritten.

Inspect in the repository: `src/domain/engine.ts`, `src/storage/local.ts`, `src/components/TankModel.tsx`, `tests/engine.test.ts` and `e2e/flows.spec.ts`.

Evidence with clear limits

Review the behaviour, the implementation and the recorded checks together.

06 Verification record

The v0.2 run on **26 September 2026** recorded passing type checks, **40 unit tests**, **12 browser tests**, a build and a production-preview check. A separately dated **5 October 2026 recheck** is documented in **docs/portfolio/VALIDATION.md**, including the actual environment, results and any setup failures.

Browser evidence includes real Canvas selection, all three complete scenario paths, persistence, rejected operations, retained safety errors, legacy-store compatibility and narrow-screen layouts. WebGL was exercised in Linux Chromium through SwiftShader software rendering.

07 A failure that changed the implementation

A viewport change exposed a Canvas sizing regression: the footer intercepted a retest click. The fix constrained the grid and Canvas container. The browser test kept a normal click, rather than forcing it through the obstruction. The failure screenshot and corrective record are retained.

08 Review in five minutes

Run. Use `npm ci` and `npm run dev` from the project folder. Open `http://127.0.0.1:5173/review/` for the English introduction.

Interact. Follow `docs/portfolio/REVIEWER_GUIDE.md` to complete the supply-valve case, inspect a score report and refresh the saved record.

Challenge. Start a new attempt, request premature removal and inspect the rejected action. Complete the case to see why a numeric score alone cannot pass.

09 Contribution and limitations

AI-assisted development: the project owner supplied the product brief and acceptance constraints. OpenAI Codex contributed substantial implementation, testing, debugging and documentation. The artefact does not imply unaided solo coding; personal contribution claims require their own evidence.

No professional repair review, learner study, physical validation or improved-learning claim is made. Native-GPU performance, every browser/device and physical printing have not been established. Local records are editable and are not formal certificates. The project demonstrates software design and verification practices, not machine learning or computational fluid dynamics.

Reading order: `README.md` → `TECHNICAL_CASE_STUDY.md` → `VALIDATION.md` → source and tests. Contribution details: `PROVENANCE.md`. All documentation filenames after `README` are under `docs/portfolio/`.